

ARC-AGI-3 Paper Track — повний детальний опис механіки гри

1. Загальна суть гри

Це покрокова навігаційна гра в лабіринті з обмеженим ресурсом життя та обов'язковою послідовністю цілей. Гравець повинен знайти найкоротший можливий маршрут, спочатку дістатися до ключа, активувати можливість завершення рівня, а потім дістатися до дверей. При цьому необхідно не допустити, щоб закінчився запас ходів або всі життя.

Основна логіка гри виглядає так:

START



ЗНАЙТИ ГРАВЦЯ НА КАРТІ



ЗНАЙТИ КЛЮЧ



ЗНАЙТИ ДВЕРІ



ЗНАЙТИ ДОДАТКОВІ ЖОВТІ БОНУСИ ЖИТТЯ



ПОБУДУВАТИ НАЙКОРОТШИЙ ВАЛІДНИЙ МАРШРУТ



ОБОВ'ЯЗКОВО ДІЙТИ ДО КЛЮЧА



ПІСЛЯ КЛЮЧА ДІЙТИ ДО ДВЕРЕЙ



ЗАВЕРШИТИ РІВЕНЬ

Ключовий принцип:

KEY → DOOR

Ключ завжди має бути отриманий раніше, ніж гравець досягне дверей як фінальної цілі.

Просто дістатися до дверей недостатньо:

START → DOOR = неправильний порядок

Правильний порядок:

START → KEY → DOOR

2. Гравець

Гравець є великою фігурою розміром **5 × 5 кубиків**.

Загалом він складається з 25 кубиків.

Його точна кольорова структура:

- дві верхні горизонтальні лінії — помаранчеві;
- три нижні горизонтальні лінії — темно-сині.

Схематично:

О О О О О

O O O O O
B B B B B
B B B B B
B B B B B

де:

O = помаранчевий кубик

B = темно-синій кубик

Таким чином:

верхня частина = 10 помаранчевих кубиків

нижня частина = 15 темно-синіх кубиків

Це важливо для розпізнавання гравця на карті. Його не потрібно визначати просто як синій або помаранчевий об'єкт. Його потрібно розпізнавати за повним патерном 5 × 5.

3. Габарити гравця потрібно завжди враховувати

Гравець не є точкою і не займає одну клітинку.

Його фактичний розмір:

WIDTH = 5

HEIGHT = 5

Тому при плануванні маршруту потрібно враховувати всю площу:

5 × 5 = 25 клітинок

Не можна просто перевірити, чи може пройти одна центральна точка.

Потрібно перевірити:

Чи може весь квадратний об'єкт розміром 5 × 5 пройти через конкретний коридор?

Наприклад, шлях може виглядати прохідним для агента розміром 1 × 1, але бути повністю непрохідним для гравця розміром 5 × 5.

Тому при кожному русі потрібно перевіряти весь footprint гравця.

Умовно:

PLAYER_FOOTPRINT ∩ WALL = Ø

Тобто жодна частина гравця не повинна перетинатися зі стіною або непрохідною областю.

4. Рух гравця

Гравець може рухатися лише у чотирьох ортогональних напрямках:

↑ ВГОРУ

↓ ВНИЗ

← ВЛІВО

→ ВПРАВО

Діагональний рух заборонений.

Не можна:

↖



При русі переміщується вся фігура гравця розміром 5 × 5.

Тобто алгоритм не повинен мислити так:

"Центральна точка може пройти"

Алгоритм повинен мислити так:

"Після переміщення всі 25 кубиків гравця повинні займати допустимі клітинки"

5. Система життя

У гравця є:

3 життя

На інтерфейсі вони показані як три червоні блоки.

Кожен червоний блок має розмір:

2 × 2 кубики

Отже, система виживання складається з двох різних рівнів ресурсу:

1. Основні життя

3 життя

Візуально:

[RED 2×2] [RED 2×2] [RED 2×2]

2. Поточна жовта смуга рухового ресурсу

Вона показує, скільки ходів ще може виконати гравець до закінчення поточного запасу.

Ці дві системи потрібно аналізувати окремо.

6. Жовта смуга життя / запас ходів

У нижній частині інтерфейсу є жовта смуга.

Вона фактично є лімітом доступних рухів.

Повна жовта смуга відповідає приблизно:

21 ходу

Кожен рух зменшує смугу на один сегмент.

Розмір одного сегмента:

2 × 2 кубики

Тобто:

1 MOVE



-1 YELLOW 2×2 SEGMENT

Якщо повна смуга містить 21 сегмент:

FULL BAR = 21 MOVES

то:

після 1 кроку = 20

після 2 кроків = 19

після N кроків = 21 - N

Отже, кожен рух має реальну ціну.

Неможливо безкінечно експериментувати або блукати лабіринтом.

Перед кожною важливою дією потрібно знати:

скільки ходів залишилося

і:

скільки ходів потрібно до наступної цілі

7. Жовті кубики життя

На деяких рівнях можуть бути присутні спеціальні жовті об'єкти.

На інших рівнях їх може не бути.

Це допоміжні об'єкти.

Їхня функція:

Повністю відновити жовту смугу життя/ходів.

Коли гравець досягає жовтого бонусу:

PLAYER → YELLOW BONUS

відбувається:

YELLOW BONUS DISAPPEARS

і:

MOVEMENT BAR = FULL

тобто:

moves_remaining = 21

або повний максимум, який відповідає повній жовтій смузі.

Жовтий об'єкт є одноразовим.

Після використання:

available = False

і він зникає з карти.

8. Як правильно використовувати жовті бонуси

Жовтий бонус не означає, що його завжди потрібно брати.

Перед початком руху необхідно визначити:

1. Де знаходиться гравець?

2. Де знаходиться ключ?

3. Де знаходяться двері?

4. Чи є жовті бонуси?

5. Де саме вони знаходяться?

6. Чи вистачає поточного запасу ходів без бонусу?

7. Чи вигідно робити detour до бонусу?

Можливі ситуації.

Варіант А — бонус не потрібен

Якщо:

START → KEY → DOOR

можна пройти в межах доступного ресурсу, жовтий бонус не потрібно

брати.

Тому що detour може зробити маршрут довшим.

Варіант В — бонус необхідний

Якщо прямий маршрут не дозволяє вижити:

START → KEY → DOOR

тоді потрібно включити жовтий бонус у маршрут.

Наприклад:

START → YELLOW → KEY → DOOR

або:

START → KEY → YELLOW → DOOR

Правильний варіант залежить від геометрії карти.

Варіант С — кілька жовтих бонусів

Якщо на карті декілька бонусів, потрібно порівняти маршрути.

Наприклад:

START → Y1 → KEY → DOOR

START → Y2 → KEY → DOOR

START → KEY → Y1 → DOOR

START → KEY → Y2 → DOOR

і вибрати той маршрут, який:

1. гарантує виживання;
2. дозволяє спочатку отримати ключ;
3. дозволяє дістатися до дверей;
4. має мінімальну кількість ходів.

9. Ключ

Ключ — це спеціальний об'єкт у формі знака плюс.

Він складається з п'яти кубиків.

Схематично:

К

К К К

К

де К — сірі або темно-сірі кубики.

Тобто структура:



Загалом:

5 кубиків

Кольори:

сірий

темно-сірий

Ключ є обов'язковою ціллю.

Його потрібно отримати перед дверима.

Коли гравець досягає ключа:

has_key = True

Це змінює стан гри.

10. Чому ключ є центральною механікою

Двері не можна використовувати як фінальну точку без ключа.

Таким чином:

has_key = False

означає:

LEVEL CANNOT BE COMPLETED THROUGH DOOR

Після отримання ключа:

PLAYER → KEY

відбувається:

has_key = True

і після цього:

PLAYER → DOOR

може завершити рівень.

Отже, у гри є обов'язкова причинно-наслідкова залежність:

KEY

↓

ACTIVATION

↓

DOOR CAN COMPLETE LEVEL

11. Двері

Двері — це фінальна ціль рівня.

Це темний структурований об'єкт.

У ньому є чотири темно-сині кубики.

Темно-синіми є:

верхній центральний

верхній правий

правий центральний

лівий нижній

Схематично:

[][B][B]

[][][B]

[B][][]

де:

B = темно-синій сегмент

Двері потрібно розпізнавати не тільки за кольором, а за структурою.

Це важливо, тому що на карті також можуть бути інші темно-сині елементи.

12. Основна послідовність проходження

Повний правильний цикл:

START

↓

ЗНАЙТИ PLAYER

↓

ЗНАЙТИ KEY

↓

ЗНАЙТИ DOOR

↓

ЗНАЙТИ YELLOW BONUSES

↓

ПРОАНАЛІЗУВАТИ HEALTH / MOVE BUDGET

↓

ПОБУДУВАТИ КАРТУ

↓

ЗНАЙТИ SHORTEST PATH ДО KEY

↓

ЗА ПОТРЕБИ ВКЛЮЧИТИ YELLOW BONUS

↓

COLLECT KEY

↓

ОНОВИТИ СТАН ГРИ

↓

ПОБУДУВАТИ SHORTEST PATH ДО DOOR

↓

ЗА ПОТРЕБИ ВИКОРИСТАТИ YELLOW BONUS

↓

REACH DOOR

↓

SUCCESS

13. Формальна модель задачі

Позначимо:

S = стартова позиція

K = ключ

D = двері

Y = множина жовтих бонусів

H = поточний запас ходів

Обов'язковий маршрут:

S → K → D

Жовті бонуси є необов'язковими:

S → Y → K → D

або:

S → K → Y → D

але тільки якщо вони необхідні або корисні.

Усі маршрути повинні виконувати:

KEY must be collected before DOOR

Player must survive

Player must fit through every corridor

No diagonal movement

Total moves should be minimized

14. Планування з урахуванням розміру 5 × 5

Це дуже важливо.

Звичайний пошук шляху працює для точки:

AGENT = 1×1

Тут:

AGENT = 5×5

Тому перед запуском shortest path алгоритму потрібно перетворити карту в карту допустимих позицій.

Позиція (x, y) є допустимою тільки якщо:

усі 25 клітинок гравця

можуть бути розташовані в допустимій області.

Умовно:

VALID_POSITION(x, y)

тільки якщо:

для всіх dx від 0 до 4

для всіх dy від 0 до 4

cell(x + dx, y + dy) є прохідною

Це означає:

коридор має бути достатньо великим для всієї фігури

а не лише для центральної точки.

15. Пошук найкоротшого маршруту

Для рівня потрібно побудувати граф.

Кожна допустима позиція гравця:

NODE

Кожен допустимий рух:

UP

DOWN

LEFT
RIGHT

є:

EDGE

Далі потрібно знайти:

shortest_path(PLOYER, KEY)

Після отримання ключа:

shortest_path(KEY, DOOR)

Якщо прямий маршрут не дозволяє вижити, потрібно додати бонус життя.

Наприклад:

**shortest_path(
 PLOYER,
 YELLOW,
 KEY,
 DOOR
)**

16. Ресурсне обмеження

Кожен маршрут повинен перевірятися не тільки за довжиною.

Потрібно перевірити:

ЧИ ВИСТАЧИТЬ ХОДІВ?

Наприклад:

distance = 18

moves_remaining = 21

Маршрут валідний.

Залишається:

3 ходи

Але якщо:

distance = 25

moves_remaining = 21

прямий маршрут невалідний без відновлення.

Тоді потрібно шукати жовтий бонус.

17. Жовтий бонус як reset

Жовтий бонус має особливу властивість.

Він не просто додає кілька ходів.

Він:

повністю відновлює жовту смугу

Отже:

before yellow:

moves_remaining = X

Після:

moves_remaining = 21

Це означає, що маршрут можна розбити на сегменти.

Наприклад:

START

↓ 12 moves

YELLOW

↓ reset to 21

KEY

↓ 9 moves

DOOR

У такому випадку потрібно оптимізувати не лише загальну відстань, але й те, де саме відбувається reset.

18. Що повинен робити агент перед першим рухом

Перед виконанням першої дії агент повинен провести повний аналіз.

Крок 1

Знайти гравця.

Шукати:

5x5

із:

2 верхніх помаранчевих рядів

3 нижніх темно-синіх рядів

Крок 2

Знайти ключ.

Шукати:

+

із п'яти сірих/темно-сірих кубиків.

Крок 3

Знайти двері.

Шукати структурований темний об'єкт із чотирма темно-синіми сегментами.

Крок 4

Знайти всі жовті бонуси.

Потрібно створити список:

Y1

Y2

Y3

...

Крок 5

Прочитати поточний ресурс.

Визначити:

скільки сегментів жовтої смуги залишилося

Крок 6

Побудувати карту.

З урахуванням:

PLAYER SIZE = 5×5

Крок 7

Знайти маршрут до ключа.

Ключ — обов'язковий.

Крок 8

Перевірити ресурс.

Чи може гравець дійти до ключа?

Якщо ні:

шукати YELLOW

Крок 9

Після ключа побудувати маршрут до дверей.

Знову перевірити ресурс.

Крок 10

Використовувати жовтий бонус тільки тоді, коли:

він необхідний для виживання

або:

він знаходиться на оптимальному маршруті

19. Оптимальна стратегія

Найкраща загальна стратегія:

1. Спочатку аналіз.

2. Потім план.

3. Потім виконання.

Не можна діяти так:

MOVE RANDOMLY

→ **observe**

→ **MOVE RANDOMLY**

→ **observe**

Тому що кожен рух витрачає ресурс.

Правильний підхід:

OBSERVE

↓

MAP

↓

OBJECT DETECTION

↓

STATE INFERENCE

↓

ROUTE PLANNING

↓

RESOURCE CHECK

↓

EXECUTE

20. Помилки, яких потрібно уникати

Помилка 1 — вважати гравця точкою

Неправильно.

Гравець:

5×5

Потрібно перевіряти всю фігуру.

Помилка 2 — ігнорувати ширину коридорів

Коридор може бути прохідним для точки, але не для квадратного об'єкта 5 × 5.

Помилка 3 — рухатися по діагоналі

Дозволені тільки:

UP

DOWN

LEFT

RIGHT

Помилка 4 — йти до дверей без ключа

Правильний порядок:

KEY → DOOR

Помилка 5 — не перевірити жовті бонуси

Перед початком маршруту завжди потрібно знати:

чи є на карті бонуси повного відновлення

Помилка 6 — автоматично брати всі бонуси

Detour може бути занадто дорогим.

Потрібно порівнювати:

вартість шляху до бонусу

із:

користю повного відновлення

Помилка 7 — не рахувати ходи

Кожен рух зменшує жовту смугу.

Тому маршрут потрібно рахувати до початку виконання.

21. Повна модель стану гри

GAME STATE:

PLAYER:

position

width = 5

height = 5

PLAYER COLORS:

top 2 rows = orange

bottom 3 rows = dark blue

SURVIVAL:

lives = 3

move_budget <= 21

MOVEMENT:

up

down

left

right

NO:

diagonal movement

OBJECTIVES:

key_collected = True / False

KEY:

plus shape

5 blocks

gray / dark gray

DOOR:

final objective
available after key

YELLOW BONUSES:

optional
one-time
disappear after use
restore full movement bar

SUCCESS:

key collected
AND door reached
AND player survives

22. Формула успішного проходження

SUCCESS =

PLAYER IDENTIFIED
AND MAP UNDERSTOOD
AND VALID 5×5 MOVEMENT
AND KEY COLLECTED
AND KEY COLLECTED BEFORE DOOR
AND RESOURCE NOT EXHAUSTED
AND DOOR REACHED

Оптимальне рішення:

OPTIMAL SOLUTION =

SUCCESS

+

MINIMUM NUMBER OF MOVES

23. Найкоротший точний опис механіки

Це покрокова задача оптимального маршруту в лабіринті для великого гравця розміром 5 × 5 клітинок. Гравець складається з двох верхніх помаранчевих рядів і трьох нижніх темно-синіх рядів. Він може рухатися лише в чотирьох напрямках — вгору, вниз, вліво та вправо — без діагонального руху. При кожному русі необхідно враховувати всю площу гравця, оскільки весь об'єкт 5 × 5 повинен поміститися в доступному просторі лабіринту.

У гравця є три життя, які відображаються трьома червоними блоками розміром 2 × 2. Окремо існує жовта смуга рухового ресурсу, повний запас

якої становить приблизно 21 хід. Кожен рух витрачає один жовтий сегмент розміром 2×2 . На деяких рівнях присутні жовті бонусні об'єкти життя. Після контакту з ними вони зникають і повністю відновлюють жовту смугу ходів.

Основна мета рівня — спочатку знайти ключ у формі плюса, який складається з п'яти сірих і темно-сірих кубиків. Після отримання ключа потрібно дістатися до дверей — фінальної точки рівня, яка має характерний темний патерн із чотирма темно-синіми сегментами. Ключ потрібно отримати раніше за двері; маршрут безпосередньо до дверей без ключа не є правильним завершенням.

Перед початком руху необхідно визначити позицію гравця, ключа, дверей, усіх жовтих бонусів життя та доступного ресурсу. Після цього потрібно побудувати карту з урахуванням реального розміру гравця 5×5 , знайти найкоротший валідний маршрут до ключа, оцінити, чи вистачає ресурсу, і за необхідності включити в маршрут жовтий бонус повного відновлення. Після отримання ключа потрібно перепланувати або продовжити маршрут до дверей.

Головна мета solver-а — не просто знайти будь-який шлях, а знайти найкоротший маршрут, який дотримується обов'язкового порядку KEY → DOOR, враховує реальні габарити гравця, не використовує діагональний рух, правильно використовує жовті відновлення і гарантує, що гравець не втратить доступний ресурс до завершення рівня.

Зе таке після активації дверей потрібно оминати оцей плюс інакше повторний прохід по ньому заблокує двері. на початку гри двері закриті. І коли проходимо перший раз над плюсом ми їх відкриваємо сам знак дверей він повертається на самому дисплеї гравця там де жовта полоса. З самого лівого кута є знак у формі ключа він має стати у такому самому напрямку як і на дверях. Причина проста кожен раз коли ми находимо на ключ (оцей от знак у формі плюса) оцей значок на панелі гравця з правої сторони повертається трошки суть у тому щоб вони дивилися в одну сторону тоді гравцеві буде дозволено пройти на наступний рівень крізь



Двері. Потрібно врахувати що кроки рахуються також. На деяких рівнях є отакі різнокольорові штуки їх потрібно теж торкатися щоб поміняти колір ключа. Також тут на стінах є оці білі лінії як на скрішоті. Вони сунуть гравця через усе поле в перед. Тактика. Оцінити спочатку що є на мапі. Далі колір ключа у правому нижньому куті і його напрямку. Далі чи є блоки жовтого кольору і чи є якісь інші предмети. Щоб знати чи потрібно їх використовувати для зміни кольору ключа і скільки ходів "туди назад" на знаку плюс потрібно щоб повернути ключ у правильному напрямку. Він кратитья лише за часовою стрілкою. Але по суті нам спочатку потрібно оцінити карту. Далі оцінити які предмети є і які нові. І подумати як їх використати у нашому агенті. Далі кількість кроків до цілі і логіку розміщення предметів. Тобто для чого їх так розмістили і що вони дають. Оцінити стан гравця і його життя і скільки кроків і доч чого у нього є.